CS6650 Homework 7 - Team Consolidation Report

Member 1 - Piyush Daga
Member 2 - Weifan Li

3. Comparative Analysis: Part II - Sync vs. Async Processing

3.1 Sync Implementation Differences & Bottlenecks

A major point of contrast in our group was how the synchronous "payment processing" 3-second delay was implemented, which drastically affected our baseline metrics.

- **Global Bottleneck (Member 1):** Implemented a global buffered channel (`make(chan struct{}, 1)`). This meant the entire system could only process exactly 1 order at a time system-wide, resulting in a hard throughput ceiling of **0.33 RPS**, regardless of the 20-user flash sale load.
- **Per-Request Bottleneck (Member 2):** Implemented a per-goroutine bottleneck (or utilized higher channel capacity). Their system queued requests concurrently within Go's HTTP server, achieving **5.74 RPS** under a 20-user load.

3.2 Async Performance & Queue Draining

Despite the differences in the Sync baseline, saw identical, massive improvements when moving to an Asynchronous (SNS -> SQS) architecture.

Metric	Member 1	Member 2 (
Sync Max Throughput	0.33 RPS	5.74 RPS
Async Max Throughput	59.5 RPS	58.82 RPS
Async Response Time (Median)	42 ms	25 ms

1 Worker Drain Time	~34 mins (3.4k queue)	~15 mins (3.2k queue)
100 Worker Drain Time	~1.7 mins	Real-time (near 0 queue)

The Async endpoint successfully decouples acceptance from processing, boosting throughput by over 10x-170x (depending on the sync bottleneck implementation) and reducing customer wait times from >30,000ms down to ~25-45ms. concluded that **180 workers** are mathematically required to process a 60 RPS flash sale in real-time (60 / 0.33 = 180).

4. Comparative Analysis: Part III - Lambda vs. ECS Workers

replaced the ECS worker polling SQS with an AWS Lambda function directly subscribed to the SNS topic (bypassing SQS).

4.1 Cold Starts

observed nearly identical cold start behaviors when SNS triggered concurrent Lambda instances:

- **Member 1:** Observed cold starts at **~70 ms**.
- **Member 2:** Observed cold starts between **73 ms - 92 ms**.
- cold start overhead is negligible (accounting for only ~2.3% of the total 3,000ms payment processing time).

4.2 Cost Analysis & Break-even Points (The Memory Factor)

two different break-even points for when ECS becomes cheaper than Lambda. By comparing reports, we discovered this was entirely due to differing Lambda memory allocations.

- **Fixed ECS Cost:** the ECS Fargate + ALB + NAT setup costs **~$17.00 / month** (always-on).
- **Member 1 (512MB Lambda):** Configured Lambda with 512MB RAM. Due to higher GB-second costs, the free tier covers ~267K orders. **Break-even: ~267,000 orders/month.**
- **Member 2 (128MB Lambda):** Configured Lambda with 128MB RAM . The free tier covers ~1.06M orders. **Break-even: 3,702,326 orders/month.**

Memory configuration is critical in Serverless. Because the Go binary is highly efficient, provisioning 512MB (Member 1) lowers the break-even threshold significantly.

By optimizing memory to 128MB (Member 2), Lambda remains the cheaper option for over 3.7 million orders per month.

5. Architectural Trade-offs

Feature	ECS + SQS Worker	Lambda (SNS Direct)
Cost (Low Traffic)	High ($17/mo fixed)	Free (Up to 1M requests/mo)
Cost (High Traffic)	Highly Efficient	Expensive (if unoptimized)
Durability/Retries	High (SQS 4-day retention)	Low (SNS retries twice, drops)
Operations	Complex (Scaling policies, VPCs)	Zero-management

Final Group Recommendation:

For an early-stage startup, **AWS Lambda is the superior choice**. It reduces infrastructure compute costs to $0 during the pre-revenue phase and automatically absorbs flash-sale spikes without manual worker tuning. The 70ms cold start is invisible to the end-user.

However, strictly advises against bypassing SQS in a production environment. Bypassing SQS strips the system of vital retry logic and visibility timeouts. A highly durable startup architecture should use **SNS -> SQS -> Lambda** to take the cost-benefits of Serverless with the durability guarantees of a queue. Once the startup exceeds ~3.5 million orders per month, the architecture should be migrated back to ECS Fargate to capitalize on flat-rate cost efficiency.